

Kyle Lin
Project 3 Question 1

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# helpers
def read_gray_pil(path):
    """Read image with PIL, convert to 8-bit grayscale, return float64 array [0..255]."""
    img = Image.open(path).convert("L") # force grayscale
    arr = np.array(img, dtype=np.float64)
    return arr

def save_gray_pil(arr, path):
    """Save a float/uint image array to disk via PIL (clipped to [0,255])."""
    arr = np.clip(arr, 0, 255).astype(np.uint8)
    Image.fromarray(arr, mode="L").save(path)

def gaussian_kernel(M, sigma=None):
    """MxM Gaussian with sum normalized to 1. M must be odd."""
    assert M % 2 == 1, "M must be odd"
    if sigma is None:
        sigma = M / 4.0
    r = M // 2
    k = np.zeros((M, M), dtype=np.float64)
    s2 = 2.0 * sigma * sigma
    for i in range(-r, r+1):
        for j in range(-r, r+1):
            k[i+r, j+r] = np.exp(-(i*i + j*j) / s2)
    s = k.sum()
    if s != 0:
        k /= s
    return k

def mean_kernel(M):
    """MxM mean filter with sum normalized to 1."""
    assert M % 2 == 1, "M must be odd"
    val = 1.0 / (M * M)
    k = np.empty((M, M), dtype=np.float64)
    for i in range(M):
        for j in range(M):
            k[i, j] = val
    return k
```

```

def convolve_interior(img, kernel):
    """
    Convolve img with kernel ONLY on interior pixels:
    interior indices i,j in [r, N-1-r]. Border is left 0.
    """
    M = kernel.shape[0]
    r = M // 2
    H, W = img.shape
    out = np.zeros_like(img, dtype=np.float64)

    for i in range(r, H - r):
        for j in range(r, W - r):
            acc = 0.0
            #  $h(i,j) = \sum_{k=0..M-1} \sum_{l=0..M-1} g[k,l] * f(i-(k-r), j-(l-r))$ 
            for k in range(M):
                for l in range(M):
                    ii = i - (k - r)
                    jj = j - (l - r)
                    acc += kernel[k, l] * img[ii, jj]
            out[i, j] = acc

    return np.clip(out, 0, 255)

# code for parameter
M = 9          # odd: 3,5,7,9,...
SIGMA = M / 4.0  # per instructions
paths = [
    "pic1grey300.jpg",
    "pic2grey300.jpg",
]

# running the code
imgs = [read_gray_pil(p) for p in paths]

g_kernel = gaussian_kernel(M, SIGMA)
m_kernel = mean_kernel(M)

gauss_out = [convolve_interior(im, g_kernel) for im in imgs]
mean_out = [convolve_interior(im, m_kernel) for im in imgs]

# display
fig, axes = plt.subplots(3, 2, figsize=(10, 12))

```

```
axes[0, 0].imshow(imgs[0], cmap="gray"); axes[0, 0].set_title("Original Image 1"); axes[0, 0].axis("off")
axes[0, 1].imshow(imgs[1], cmap="gray"); axes[0, 1].set_title("Original Image 2"); axes[0, 1].axis("off")
```

```
axes[1, 0].imshow(gauss_out[0], cmap="gray"); axes[1, 0].set_title(f"pic1 - Gaussian filter");
axes[1, 0].axis("off")
axes[1, 1].imshow(gauss_out[1], cmap="gray"); axes[1, 1].set_title(f"pic2 - Gaussian filter");
axes[1, 1].axis("off")
```

```
axes[2, 0].imshow(mean_out[0], cmap="gray"); axes[2, 0].set_title(f"pic1 - Mean filter"); axes[2, 0].axis("off")
axes[2, 1].imshow(mean_out[1], cmap="gray"); axes[2, 1].set_title(f"pic2 - Mean filter"); axes[2, 1].axis("off")
```

```
plt.tight_layout()
plt.show()
```

Original Image 1



pic1 - Gaussian filter



pic1 - Mean filter



Original Image 2



pic2 - Gaussian filter



pic2 - Mean filter



Project 3 Question 2

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# 1D gaussian (sum = 1)
def gaussian_filter_1d(M: int, sigma: float) -> np.ndarray:
    assert M % 2 == 1, "M must be odd"
    half = M // 2
    k = np.arange(M) - half
    g = np.exp(-(k**2) / (2.0 * sigma * sigma))
    g /= g.sum()
    return g.astype(np.float32)

# row then column filtering (interior only, borders left 0)
def filter_rows(image: np.ndarray, g: np.ndarray) -> np.ndarray:
    M = g.size
    H, W = image.shape
    pad = M // 2
    h1 = np.zeros_like(image, dtype=np.float32)
    for i in range(H):
        for j in range(pad, W - pad):
            s = 0.0
            for k in range(M):
                s += g[k] * image[i, j - (k - pad)]
            h1[i, j] = s
    return h1

def filter_columns(h1: np.ndarray, g: np.ndarray) -> np.ndarray:
    M = g.size
    H, W = h1.shape
    pad = M // 2
    h2 = np.zeros_like(h1, dtype=np.float32)
    for j in range(W):
        for i in range(pad, H - pad):
            s = 0.0
            for k in range(M):
                s += g[k] * h1[i - (k - pad), j]
            h2[i, j] = s
    return h2

# gradient magnitude and contrast stretch
def gradient_mag_0_255(img: np.ndarray) -> np.ndarray:
```

```

gx = np.zeros_like(img, dtype=np.float32)
gy = np.zeros_like(img, dtype=np.float32)
gx[:-1, :] = img[1:, :] - img[:-1, :]
gy[:, :-1] = img[:, 1:] - img[:, :-1]
mag = np.sqrt(gx * gx + gy * gy)

# zero borders explicitly
mag[0, :] = mag[-1, :] = 0
mag[:, 0] = mag[:, -1] = 0

# stretch to 0 - 255 using the 99th percentile to avoid outliers
p99 = np.percentile(mag, 99.0)
scale = 255.0 / (p99 + 1e-6)
mag = np.clip(mag * scale, 0, 255)
return mag.astype(np.float32)

```

```

def threshold_edges(mag_0255: np.ndarray, T: float) -> np.ndarray:
    out = np.zeros_like(mag_0255, dtype=np.uint8)
    out[mag_0255 > T] = 255
    return out

```

```

def run_one(img: np.ndarray, sigma: float, thresholds):
    M = 9 # assignment: 9x9 gaussian
    g = gaussian_filter_1d(M, sigma)
    h1 = filter_rows(img, g)
    h2 = filter_columns(h1, g)
    mag = gradient_mag_0_255(h2) # normalized to 0 - 255

```

```

fig, axes = plt.subplots(2, 2, figsize=(8, 8))
for ax, T in zip(axes.ravel(), thresholds):
    edges = threshold_edges(mag, T)
    ax.imshow(edges, cmap="gray")
    ax.set_title(f" $\sigma$ ={sigma}, T={T}")
    ax.axis("off")
fig.tight_layout()
plt.show()

```

```

img1 = np.array(Image.open("pic1grey300.jpg").convert("L"), dtype=np.float32)
img2 = np.array(Image.open("pic2grey300.jpg").convert("L"), dtype=np.float32)

```

```

sigmas = [1.0, 2.0, 3.0]
thresholds = [20, 30, 40, 50]

```

```

for s in sigmas:

```

```
run_one(img1, s, thresholds)
```

```
for s in sigmas:
```

```
    run_one(img2, s, thresholds)
```

$\sigma=1.0$, $T=20$



$\sigma=1.0$, $T=30$



$\sigma=1.0$, $T=40$



$\sigma=1.0$, $T=50$



$\sigma=2.0, T=20$



$\sigma=2.0, T=30$



$\sigma=2.0, T=40$



$\sigma=2.0, T=50$



$\sigma=3.0$, $T=20$



$\sigma=3.0$, $T=30$



$\sigma=3.0$, $T=40$



$\sigma=3.0$, $T=50$



$\sigma=1.0, T=20$



$\sigma=1.0, T=30$



$\sigma=1.0, T=40$



$\sigma=1.0, T=50$



$\sigma=2.0$, $T=20$



$\sigma=2.0$, $T=30$



$\sigma=2.0$, $T=40$



$\sigma=2.0$, $T=50$



$\sigma=3.0, T=20$



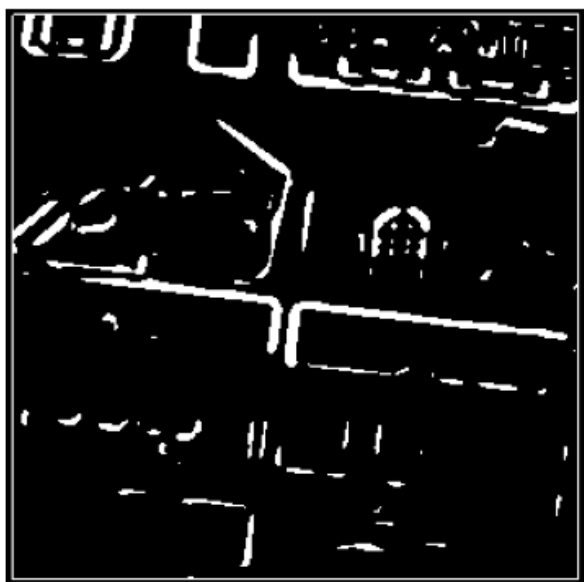
$\sigma=3.0, T=30$



$\sigma=3.0, T=40$



$\sigma=3.0, T=50$



Project 3 Question 3a

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# helpers
def gaussian_kernel_1d(length: int, sigma: float) -> np.ndarray:
    assert length % 2 == 1, "Kernel length must be odd"
    half = length // 2
    k = np.arange(length) - half
    g = np.exp(-(k**2) / (2.0 * sigma * sigma))
    g /= g.sum()
    return g.astype(np.float32)

def separable_filter(img: np.ndarray, k: np.ndarray) -> np.ndarray:
    H, W = img.shape
    M = k.size
    pad = M // 2
    tmp = np.zeros_like(img, dtype=np.float32)
    out = np.zeros_like(img, dtype=np.float32)
    # rows
    for i in range(H):
        for j in range(pad, W - pad):
            tmp[i, j] = float(np.dot(k, img[i, j - pad:j + pad + 1]))
    # cols
    for j in range(W):
        for i in range(pad, H - pad):
            out[i, j] = float(np.dot(k, tmp[i - pad:i + pad + 1, j]))
    return out

# gradients
def gradients_forward(img: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    lx = np.zeros_like(img, dtype=np.float32)
    ly = np.zeros_like(img, dtype=np.float32)
    lx[:-1, :] = img[1:, :] - img[:-1, :]
    ly[:, :-1] = img[:, 1:] - img[:, :-1]
    lx[0, :] = lx[-1, :] = 0
    ly[:, 0] = ly[:, -1] = 0
    lx /= 10.0
    ly /= 10.0
    return lx, ly

def harris_R(lx: np.ndarray, ly: np.ndarray) -> np.ndarray:
```

```

A = lx * lx
B = ly * ly
C = lx * ly
k11 = gaussian_kernel_1d(11, 5.5)
A_ = separable_filter(A, k11)
B_ = separable_filter(B, k11)
C_ = separable_filter(C, k11)
detM = A_ * B_ - C_ * C_
traceM = A_ + B_
R = detM - 0.04 * (traceM * traceM)
R[0, :] = R[-1, :] = 0
R[:, 0] = R[:, -1] = 0
return R

```

```

# fixed 3x3 nms
def nms_3x3(R: np.ndarray) -> np.ndarray:
    H, W = R.shape
    Rp = np.pad(R, 1, mode="constant", constant_values=-np.inf)
    neighbor_max = np.full((H, W), -np.inf, dtype=np.float32)
    for di in (-1, 0, 1):
        for dj in (-1, 0, 1):
            if di == 0 and dj == 0: # skip center
                continue
            nb = Rp[1+di:1+di+H, 1+dj:1+dj+W]
            neighbor_max = np.maximum(neighbor_max, nb)
    return R > neighbor_max

```

```

# main, it returns 3 values
def harris_corners(img_uint8: np.ndarray, thresh_frac: float = 0.08):
    img = img_uint8.astype(np.float32)
    k9 = gaussian_kernel_1d(9, 2.0) # 9x9, sigma=2
    sm = separable_filter(img, k9)
    lx, ly = gradients_forward(sm)
    R = harris_R(lx, ly)
    T = float(R.max()) * thresh_frac if R.max() > 0 else 0.0
    peaks = (R > T) & nms_3x3(R)

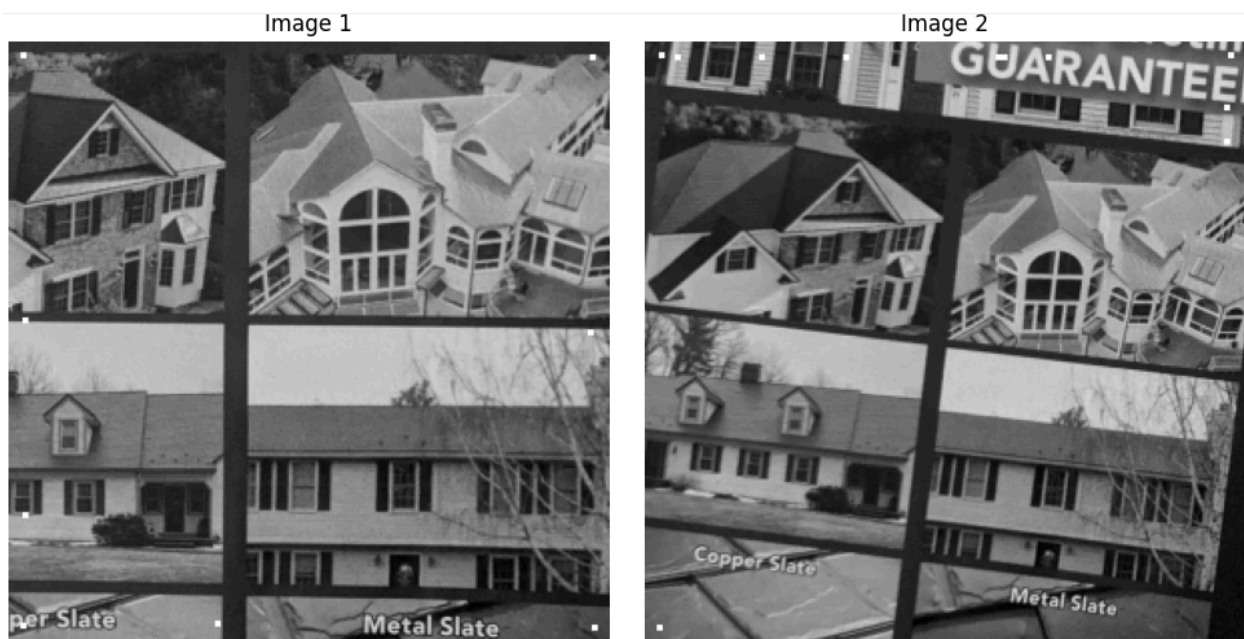
    out = img_uint8.copy()
    ys, xs = np.where(peaks)
    for y, x in zip(ys, xs):
        out[max(0, y-1):min(out.shape[0], y+2),
            max(0, x-1):min(out.shape[1], x+2)] = 255
    print(f"Max R={R.max():.3f} | T={T:.3f} | corners={len(xs)}")
    return out, peaks, R # <- THREE values

```

```
img1 = np.array(Image.open("pic1grey300.jpg").convert("L"))
img2 = np.array(Image.open("pic2grey300.jpg").convert("L"))
```

```
vis1, peaks1, R1 = harris_corners(img1, thresh_frac=0.08)
vis2, peaks2, R2 = harris_corners(img2, thresh_frac=0.08)
```

```
fig, axs = plt.subplots(1,2,figsize=(10,5))
axs[0].imshow(vis1, cmap="gray"); axs[0].set_title("Image 1"); axs[0].axis("off")
axs[1].imshow(vis2, cmap="gray"); axs[1].set_title("Image 2"); axs[1].axis("off")
plt.tight_layout(); plt.show()
```



Project 3 Question 3b

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

# helpers
def gaussian_kernel_1d(length: int, sigma: float) -> np.ndarray:
    assert length % 2 == 1, "Kernel length must be odd"
    half = length // 2
    k = np.arange(length) - half
    g = np.exp(-(k**2) / (2.0 * sigma * sigma))
    g /= g.sum()
    return g.astype(np.float32)

def separable_filter(img: np.ndarray, k: np.ndarray) -> np.ndarray:
    H, W = img.shape
    M = k.size
    pad = M // 2
    tmp = np.zeros_like(img, dtype=np.float32)
    out = np.zeros_like(img, dtype=np.float32)
    # rows
    for i in range(H):
        for j in range(pad, W - pad):
            tmp[i, j] = float(np.dot(k, img[i, j - pad:j + pad + 1]))
    # cols
    for j in range(W):
        for i in range(pad, H - pad):
            out[i, j] = float(np.dot(k, tmp[i - pad:i + pad + 1, j]))
    return out

# gradients (forward diff; divide by 10 per spec)
def gradients_forward(img: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    H, W = img.shape
    lx = np.zeros_like(img, dtype=np.float32)
    ly = np.zeros_like(img, dtype=np.float32)
    lx[:-1, :] = img[1:, :] - img[:-1, :]
    ly[:, :-1] = img[:, 1:] - img[:, :-1]
    lx[0, :] = lx[-1, :] = 0
    ly[:, 0] = ly[:, -1] = 0
    lx /= 10.0
    ly /= 10.0
    return lx, ly
```

```
# harris response and NMS
```

```
def harris_R(lx: np.ndarray, ly: np.ndarray) -> np.ndarray:
```

```
    A = lx * lx
    B = ly * ly
    C = lx * ly
    k11 = gaussian_kernel_1d(11, 5.5)
    A_ = separable_filter(A, k11)
    B_ = separable_filter(B, k11)
    C_ = separable_filter(C, k11)
    detM = A_ * B_ - C_ * C_
    traceM = A_ + B_
    R = detM - 0.04 * (traceM * traceM)
    R[0, :] = R[-1, :] = 0
    R[:, 0] = R[:, -1] = 0
    return R
```

```
def nms_3x3(R: np.ndarray) -> np.ndarray:
```

```
    H, W = R.shape
    Rp = np.pad(R, 1, mode="constant", constant_values=-np.inf)
    neighbor_max = np.full((H, W), -np.inf, dtype=np.float32)
    for di in (-1, 0, 1):
        for dj in (-1, 0, 1):
            if di == 0 and dj == 0:
                continue
            nb = Rp[1+di:1+di+H, 1+dj:1+dj+W]
            neighbor_max = np.maximum(neighbor_max, nb)
    return R > neighbor_max
```

```
def harris_corners_coords(img_uint8: np.ndarray, thresh_frac: float = 0.08):
```

```
    img = img_uint8.astype(np.float32)
    # initial smoothing 9x9  $\sigma=2$ 
    k9 = gaussian_kernel_1d(9, 2.0)
    sm = separable_filter(img, k9)
    lx, ly = gradients_forward(sm)
    R = harris_R(lx, ly)
    T = float(R.max()) * thresh_frac if R.max() > 0 else 0.0
    peaks = (R > T) & nms_3x3(R)
    ys, xs = np.where(peaks)
    print(f'Harris: Max R={R.max():.3f} | T={T:.3f} | corners={len(xs)}')
    return (ys, xs), R, lx, ly
```

```
# descriptor over 9x9 window
```

```
def descriptor_9x9_at(ys, xs, lx, ly):
```

```
    """
```

for each (y,x) corner, compute 8-bin histogram of gradient directions within 9x9 window centered at (y,x). Bins at 0,45, ... ,315 deg; voting by magnitude. then rotate so max bin lands at 180 deg (bin index 4).
returns: list of (y, x, hist_rotated(np.ndarray length 8))

```

"""
H, W = lx.shape
results = []
r = 4 # radius for 9x9
# precompute angle and magnitude images
mag = np.sqrt(lx*lx + ly*ly)
ang = (np.degrees(np.arctan2(ly, lx)) + 360.0) % 360.0 # [0,360)
for y, x in zip(ys, xs):
    # skip corners too close to border (need full 9x9)
    if y - r < 0 or y + r >= H or x - r < 0 or x + r >= W:
        continue
    h = np.zeros(8, dtype=np.float32)
    # accumulate votes
    for yy in range(y - r, y + r + 1):
        for xx in range(x - r, x + r + 1):
            a = ang[yy, xx]
            m = mag[yy, xx]
            bin_idx = int(np.round(a / 45.0)) % 8
            h[bin_idx] += m
    # rotate so max moves to 180 degrees -> bin 4
    m_idx = int(np.argmax(h))
    shift = (4 - m_idx) % 8
    h_rot = np.roll(h, shift)
    results.append((y, x, h_rot))
return results

```

```

def print_descriptors(results):
    for (y, x, h) in results:
        # format to one decimal place like the example
        vals = ", ".join(f"{v:.1f}" for v in h.tolist())
        print(f"pixel at (i,j)=(y,x) has histogram {vals}")

```

```

# load image(s), detect corners, compute and print descriptors
if __name__ == "__main__":
    img1 = np.array(Image.open("pic1grey300.jpg").convert("L"))
    img2 = np.array(Image.open("pic2grey300.jpg").convert("L"))

```

```

# get corners
(ys1, xs1), R1, lx1, ly1 = harris_corners_coords(img1, thresh_frac=0.06)
(ys2, xs2), R2, lx2, ly2 = harris_corners_coords(img2, thresh_frac=0.06)

```

```
# compute descriptors
desc1 = descriptor_9x9_at(ys1, xs1, lx1, ly1)
desc2 = descriptor_9x9_at(ys2, xs2, lx2, ly2)
```

```
print("\n Image 1 descriptors ")
print_descriptors(desc1)
```

```
print("\n Image 2 descriptors ")
print_descriptors(desc2)
```

```
Harris: Max R=125.891 | T=7.553 | corners=8
Harris: Max R=121.945 | T=7.317 | corners=14
```

Image 1 descriptors

```
pixel at (i,j)=(7,7) has histogram 0.5, 0.2, 0.0, 1.1, 93.6, 5.0, 81.4, 6.6
pixel at (i,j)=(8,291) has histogram 0.1, 1.0, 0.4, 0.0, 78.2, 10.7, 41.0, 0.1
pixel at (i,j)=(139,8) has histogram 10.3, 0.0, 0.0, 0.0, 101.7, 0.7, 2.6, 4.8
pixel at (i,j)=(145,290) has histogram 0.0, 0.0, 0.0, 10.0, 97.3, 0.0, 0.0, 0.0
pixel at (i,j)=(236,8) has histogram 0.0, 0.0, 0.0, 1.2, 91.7, 2.3, 0.0, 0.0
pixel at (i,j)=(290,104) has histogram 0.0, 0.0, 0.1, 0.1, 90.7, 0.3, 0.0, 0.0
pixel at (i,j)=(291,7) has histogram 0.9, 2.2, 6.5, 6.3, 120.4, 4.7, 115.3, 11.6
pixel at (i,j)=(292,292) has histogram 0.0, 0.0, 0.0, 0.0, 53.1, 11.0, 52.0, 0.0
```

Image 2 descriptors

```
pixel at (i,j)=(7,8) has histogram 0.0, 0.0, 65.1, 0.6, 72.3, 2.8, 0.0, 0.0
pixel at (i,j)=(7,292) has histogram 7.1, 6.1, 96.0, 0.0, 134.5, 0.3, 0.4, 7.8
pixel at (i,j)=(8,16) has histogram 42.0, 0.0, 0.1, 0.3, 50.2, 0.0, 0.0, 0.0
pixel at (i,j)=(8,58) has histogram 38.0, 0.1, 0.0, 0.2, 51.6, 0.0, 0.0, 0.0
pixel at (i,j)=(8,100) has histogram 9.7, 0.0, 0.0, 0.1, 103.6, 0.0, 0.0, 0.0
pixel at (i,j)=(8,125) has histogram 11.8, 0.5, 0.0, 1.5, 50.4, 14.7, 12.2, 9.6
pixel at (i,j)=(8,156) has histogram 18.9, 8.8, 3.2, 16.1, 30.1, 12.6, 7.1, 5.3
pixel at (i,j)=(8,176) has histogram 28.5, 1.1, 0.2, 2.5, 46.5, 5.3, 2.2, 9.1
pixel at (i,j)=(8,179) has histogram 31.1, 5.3, 2.6, 13.1, 42.8, 0.6, 0.2, 2.5
pixel at (i,j)=(8,184) has histogram 33.6, 0.0, 0.0, 0.0, 39.7, 1.4, 1.4, 16.2
pixel at (i,j)=(8,201) has histogram 41.3, 8.3, 0.7, 4.0, 48.3, 0.0, 0.0, 0.0
pixel at (i,j)=(33,290) has histogram 46.5, 5.7, 0.0, 2.9, 50.9, 0.2, 0.0, 0.0
pixel at (i,j)=(50,290) has histogram 0.0, 0.0, 2.4, 4.1, 87.5, 18.4, 13.4, 0.0
pixel at (i,j)=(292,7) has histogram 0.0, 0.0, 0.0, 0.0, 92.9, 1.9, 90.6, 0.0
```